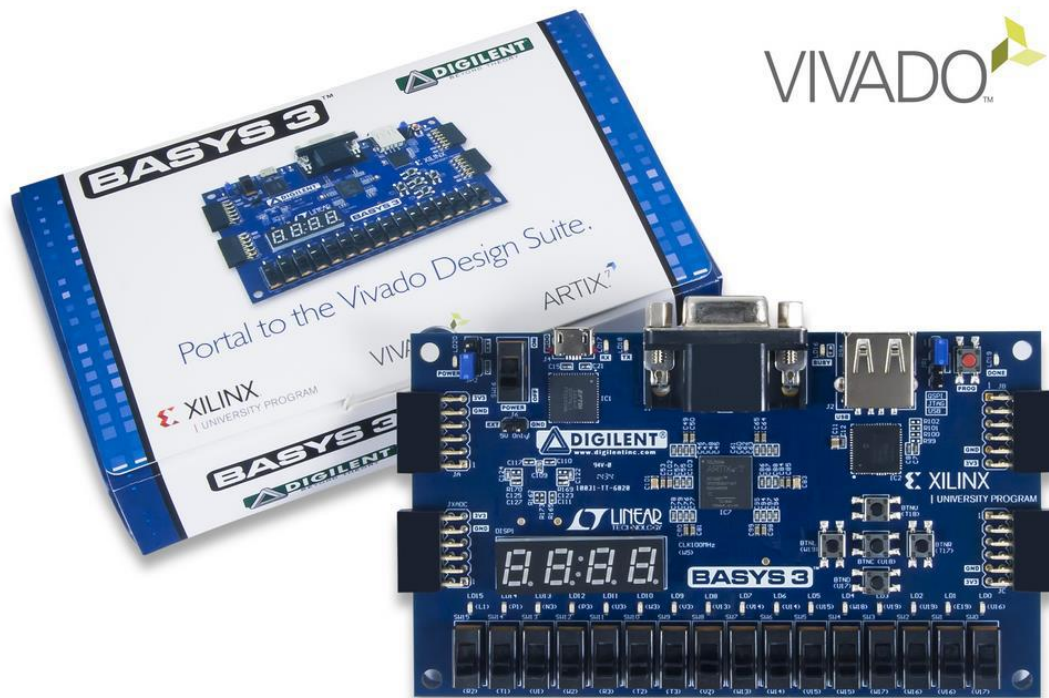




## **DIGILENT Basys 3 FPGA 基于 Vivado 上手教程 (二) Vivado IP 集成器设计环境**

## Vivado IP 集成器设计环境

### 一、 实验目的

通过将已经完成的 RTL 设计封装成可以调用的模块(IP 核), 并通过模块化设计调用 IP 核熟悉 Vivado IP 集成器的设计环境。

### 二、 实验内容

- 添加 HDL 文件进行模块设计
- 将 HDL 设计文件封装成 IP 核
- 通过模块化设计(Block Design)新建工程项目
- 添加引脚约束并综合实现设计
- 在 Basys 3 FPGA 开发板上运行实验设计

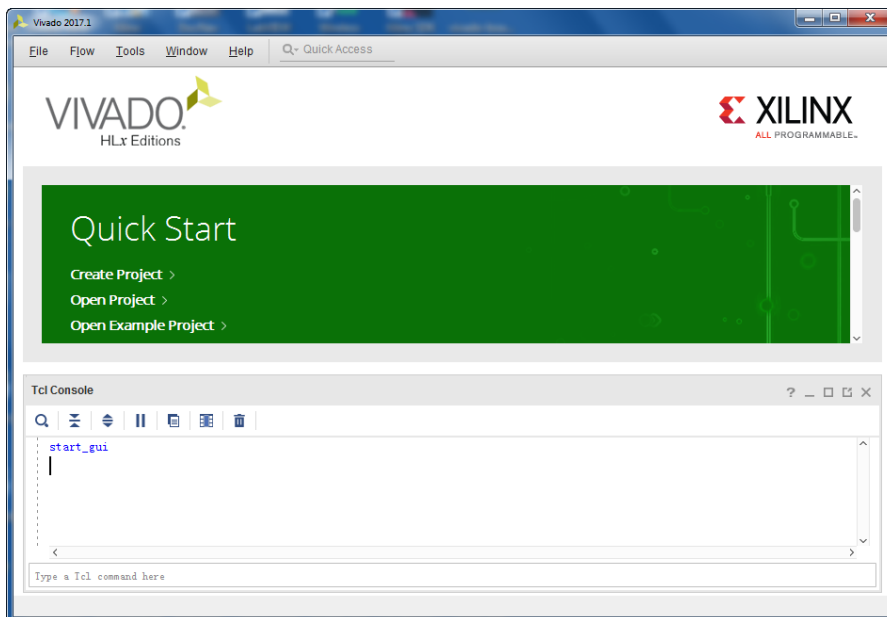
### 三、 实验步骤

#### 1. 创建新的工程项目

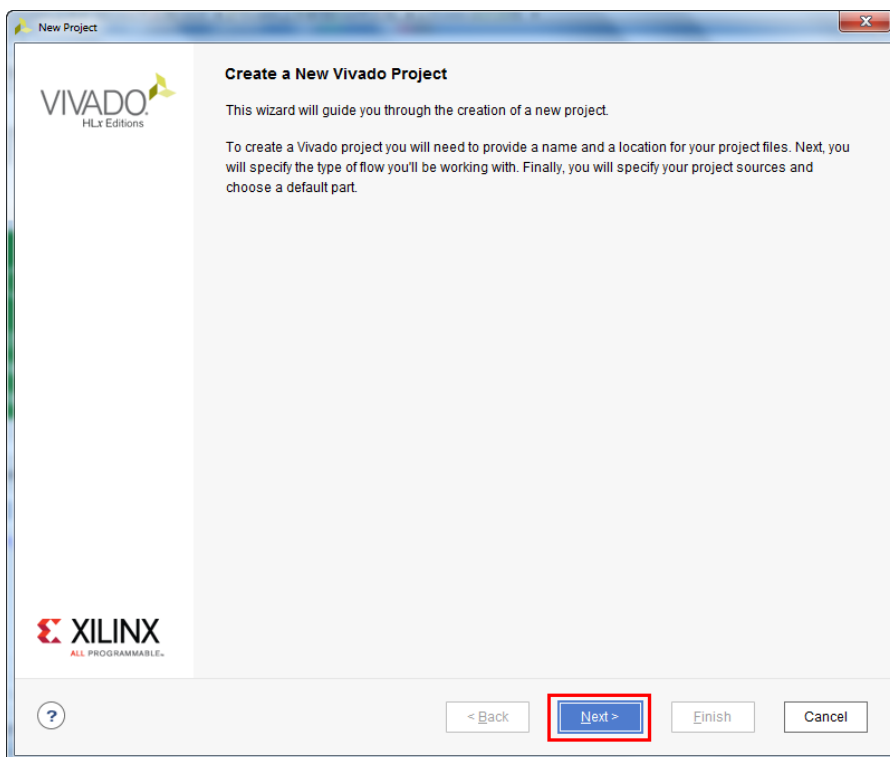
- 1) 双击桌面图标打开 Vivado 2017.1, 或者选择开始>所有程序>Xilinx Design Tools> Vivado 2017.1>Vivado 2017.1



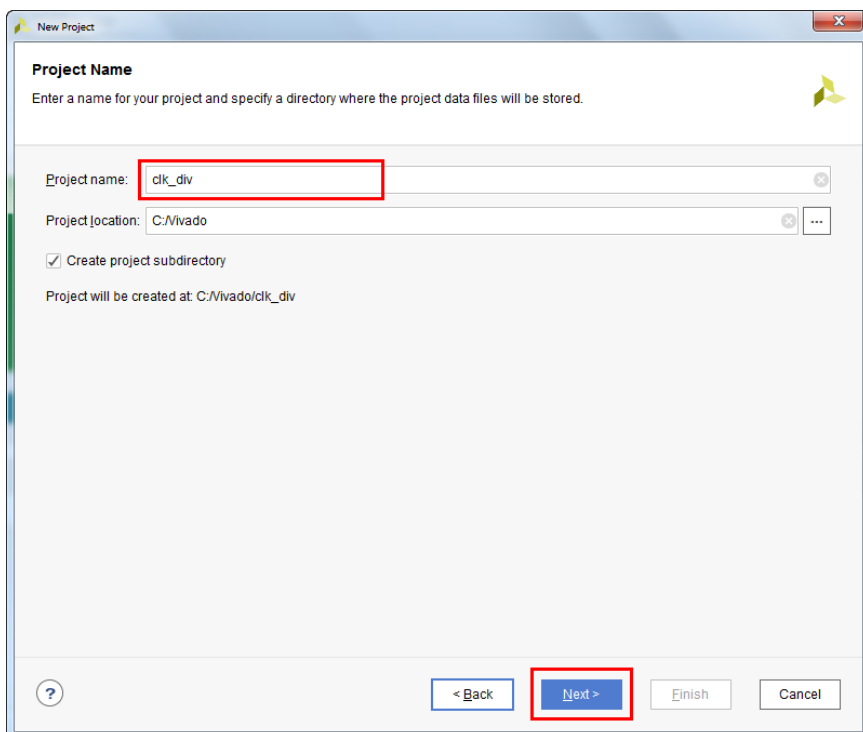
- 2) 点击‘Create Project’, 或者单击 File>New Project 创建工程文件。



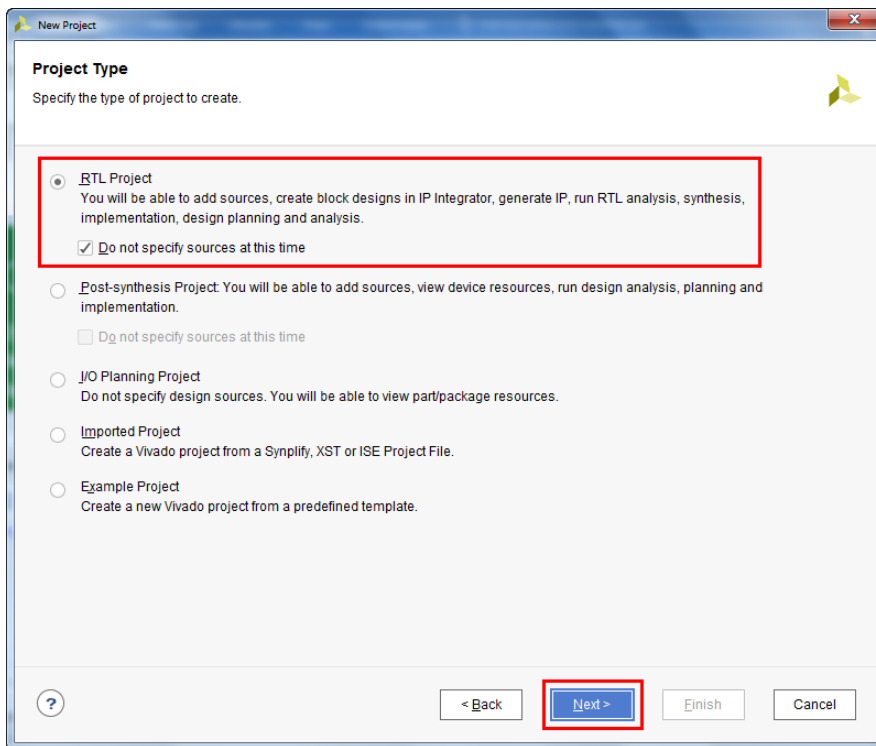
3) 弹出工程导向窗口，点击 **Next** 继续。



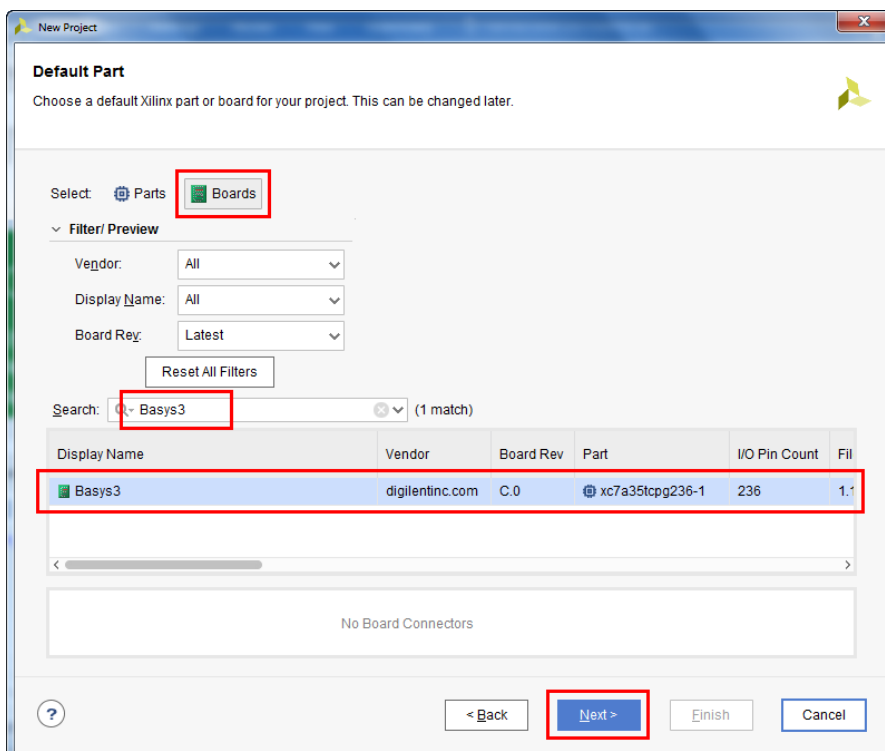
4) 将新的工程项目命名为'clk\_div'，勾选创建工程子文件夹，点击 **Next** 继续。



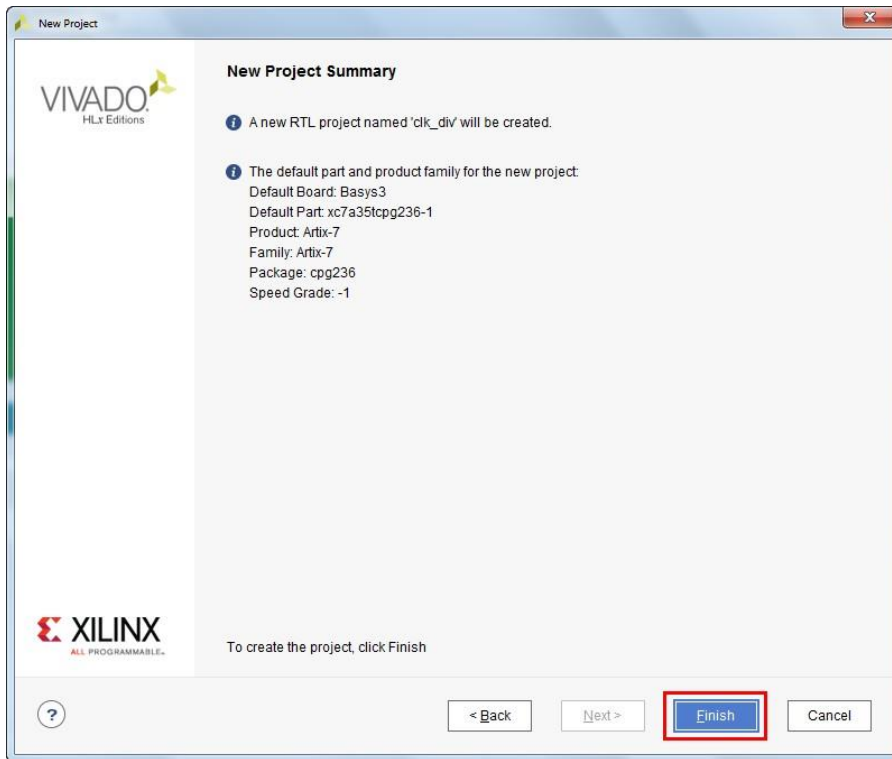
- 5) 新建一个RTL工程，勾选‘Do not specify sources at this time’先不添加源文件，点击 Next 继续。



- 6) 点击 Board，在搜索框输入‘Basys3’，选择 Basys3 开发板，点击 Next 继续。注意需要先安装 Digilent board files。

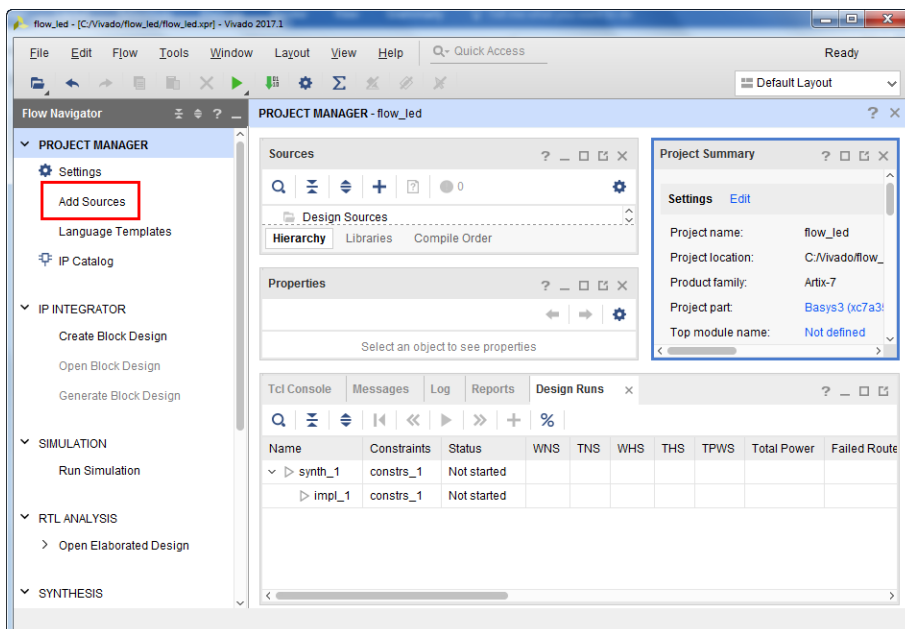


7) 确认相关信息无误后点击 **Finish** 完成工程项目的创建。

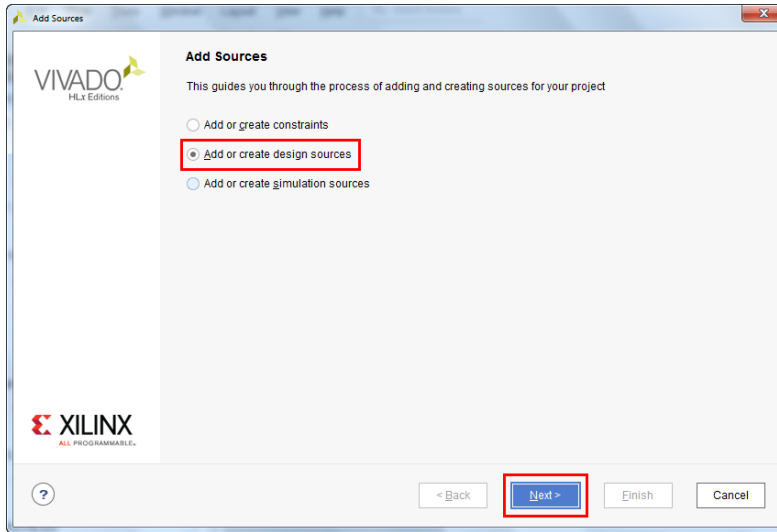


## 2. 添加源文件

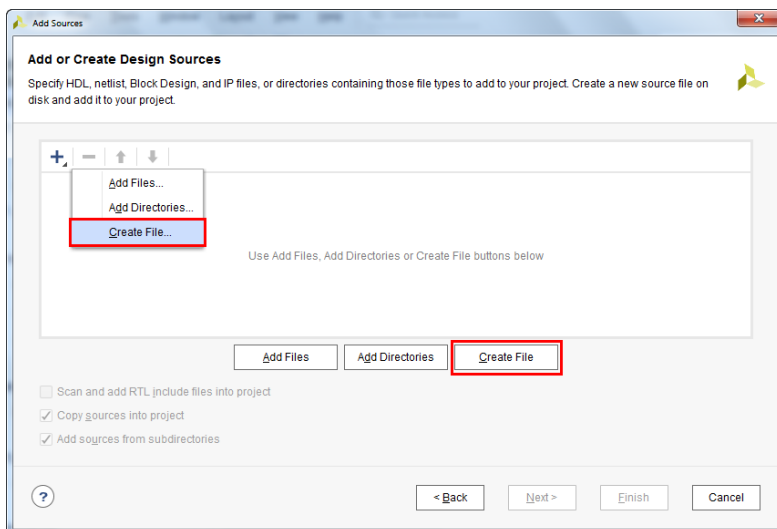
1) 完成工程创建后会弹出如下界面，在 **Flow Navigator** 中展开 **Project Manger**，点击 **Add Source** 添加源文件。



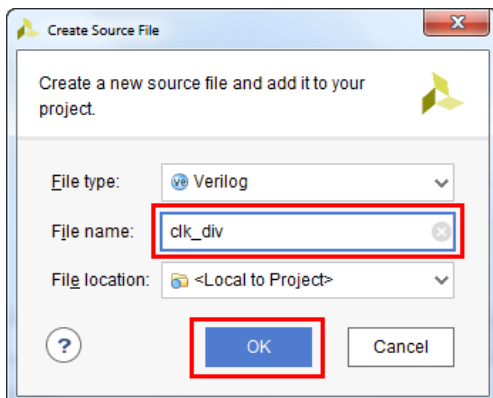
2) 选择‘Add or create design source’，点击 Next 继续。



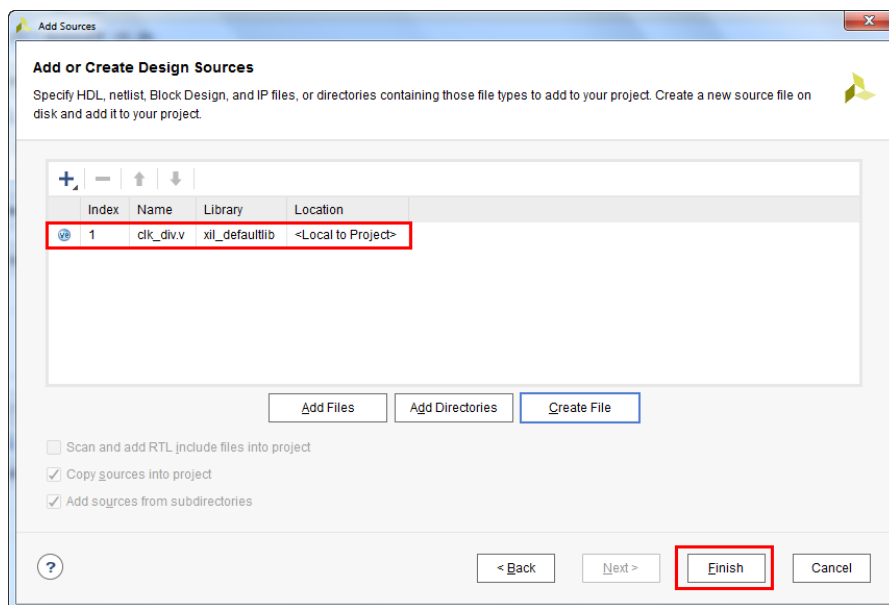
3) 点击 Create File 或者点击‘+’号，选择‘Create File’。



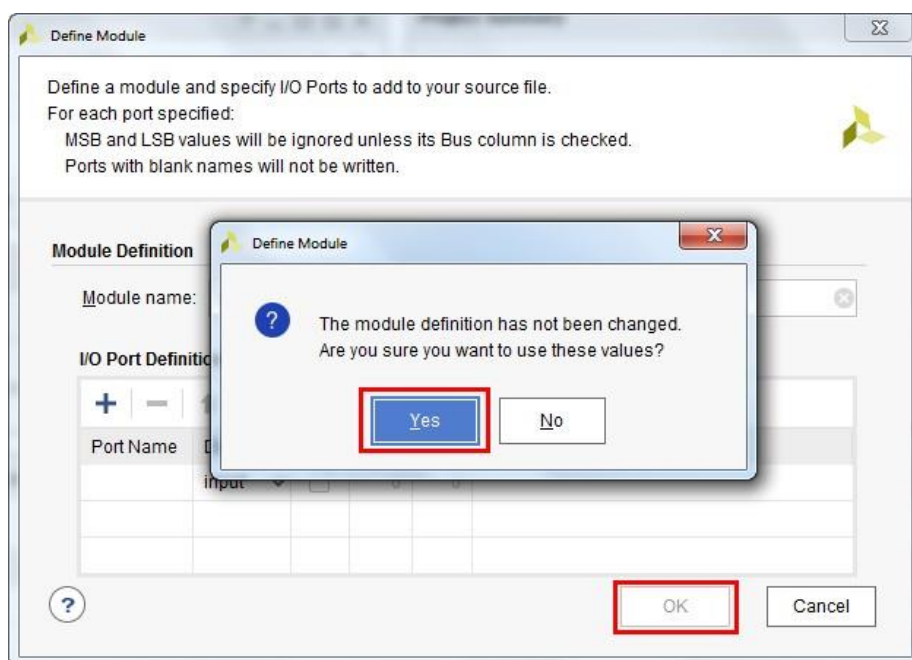
4) 在弹出的窗口中输入文件名‘clk\_div’，点击 OK。



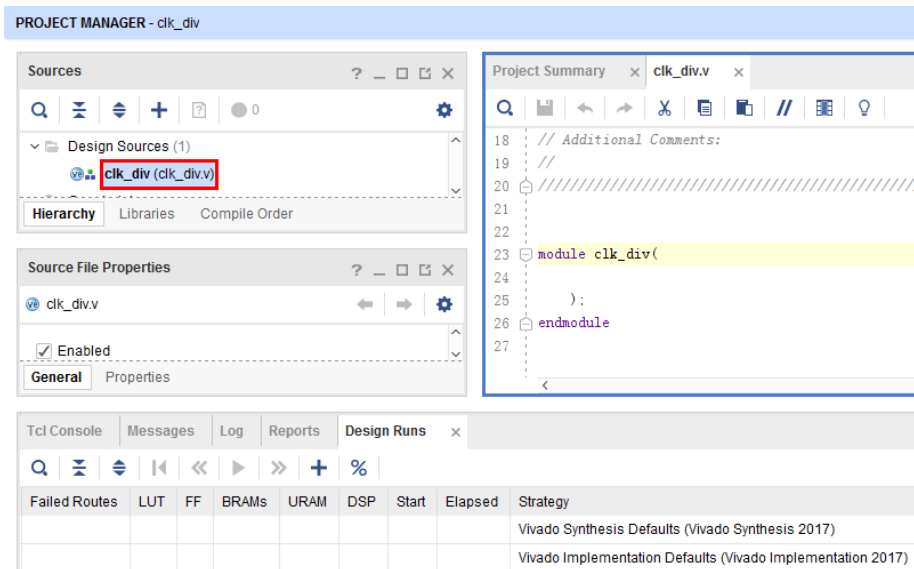
5) 新的源文件已经被添加到列表中，点击 **Finish** 完成添加。



6) 在定义输入/输出端口对话框中点击 **OK**，在弹出窗口中点击 **Yes**。



7) 在 Source 中展开 Design Sources，双击 clk\_div 打开‘clk\_div.v’文件。



8) 在‘clk\_div.v’文件中需要完成模块的功能设计，在文件中输入相应的代码。  
参考代码：

```
`timescale 1ns / 1ps
module clk_div(
    clk,          //100MHz
    clk_sys       //1Hz
);

    input clk;
    output clk_sys;

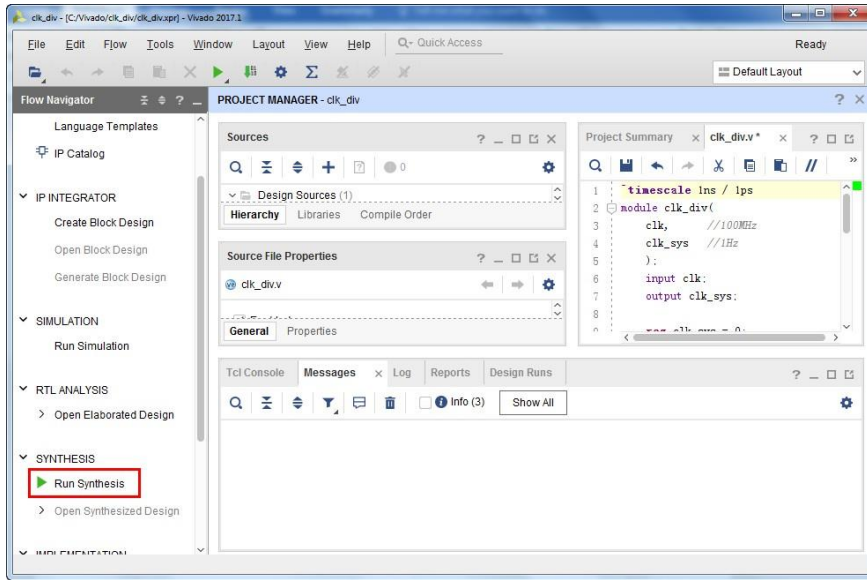
    reg clk_sys = 0;
    reg[25:0] div_counter = 0;

    always@(posedge clk)
    begin
        if(div_counter>=50000000)
        begin
            clk_sys<=~clk_sys;
            div_counter<=0;
        end
        else begin
            div_counter<=div_counter+1;
        end
    end
endmodule
```

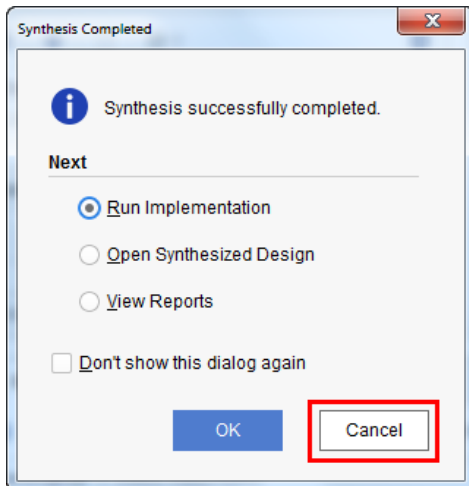
输入完成后 **Ctrl+S** 保存。



9) 在 Flow Navigator 中点击‘Run Synthesis’对工程进行综合。

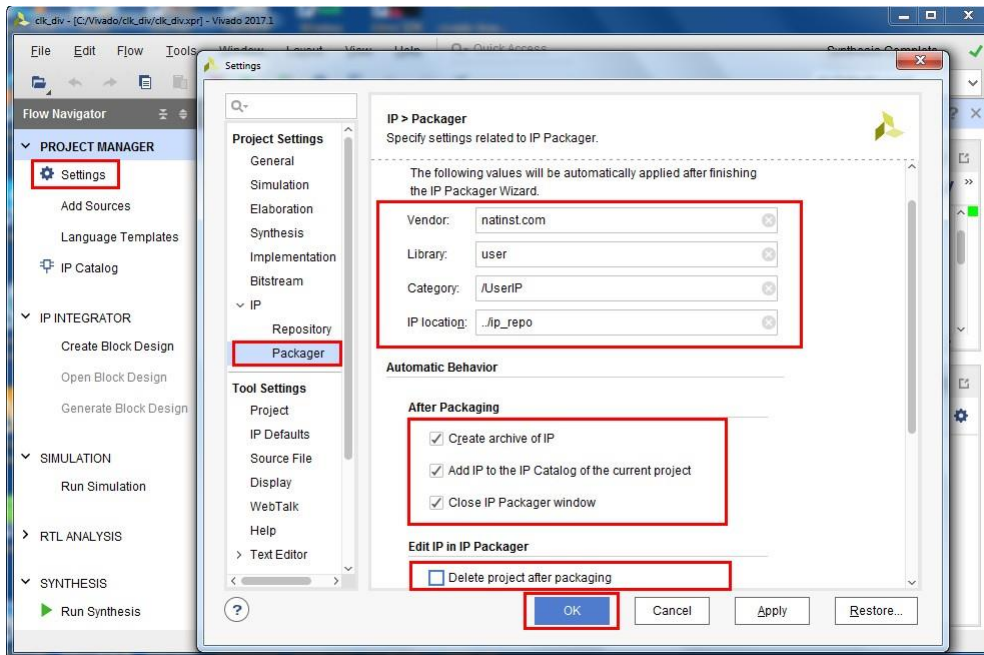


10) 综合完成后，点击‘Cancel’关闭弹出的对话框。

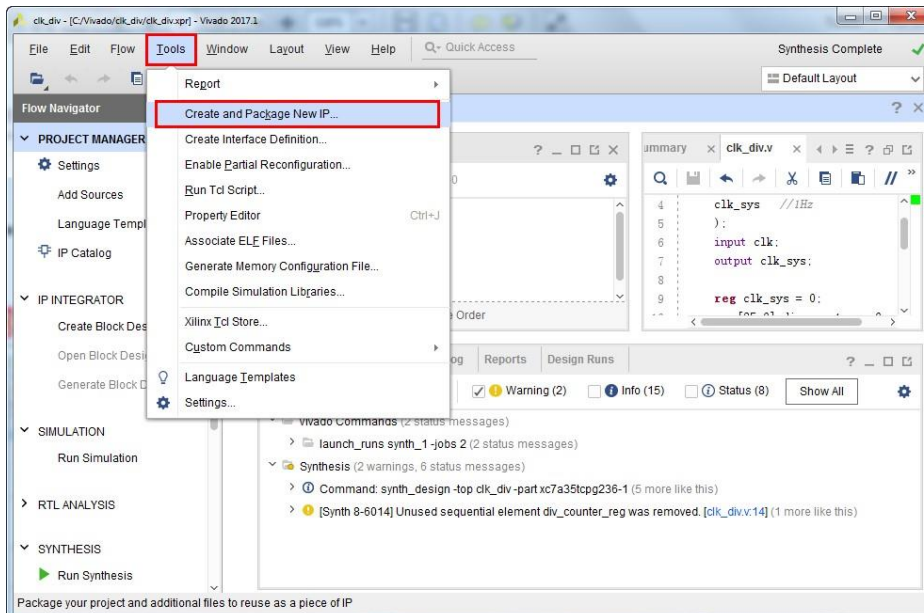


### 3. IP 核封装

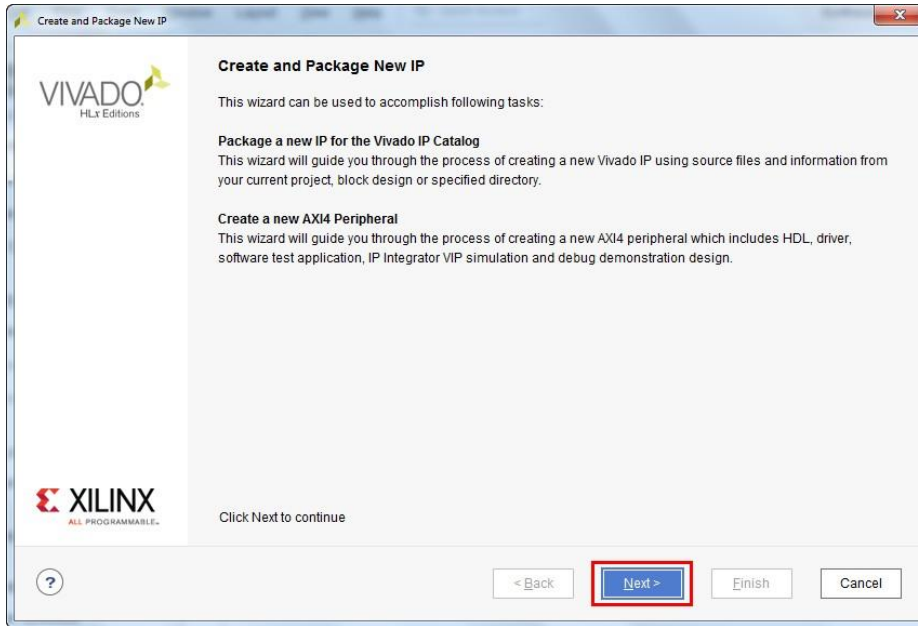
- 1) 在界面左侧的 Flow Navigator 中展开 Project Manager，点击 Settings，在 Project Settings 中展开 IP 选项，点击‘Packager’。根据下图填写对话框中的内容并完成相应的勾选，点击 OK 继续。



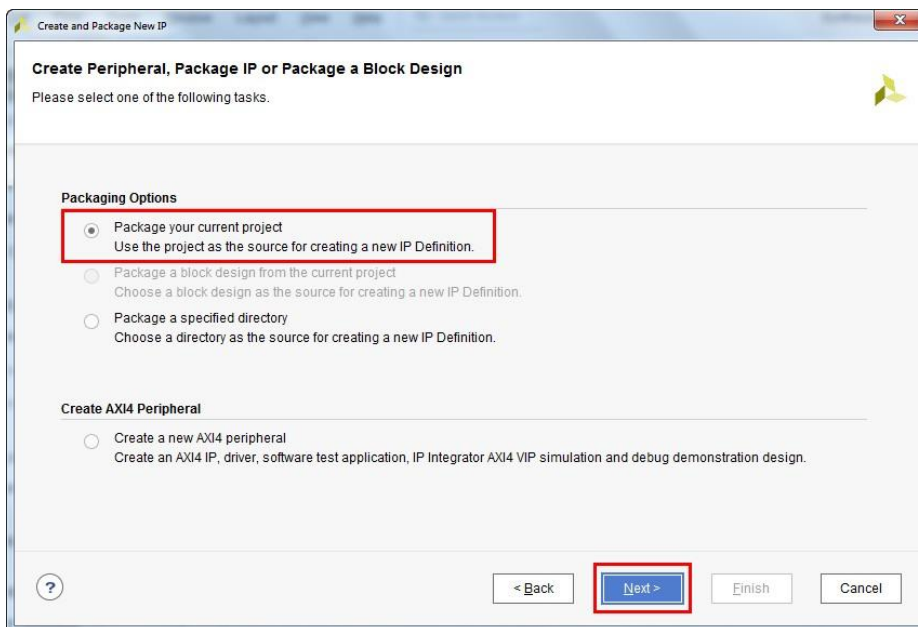
- 2) 点击菜单栏中 Tools>Create and Package New IP...



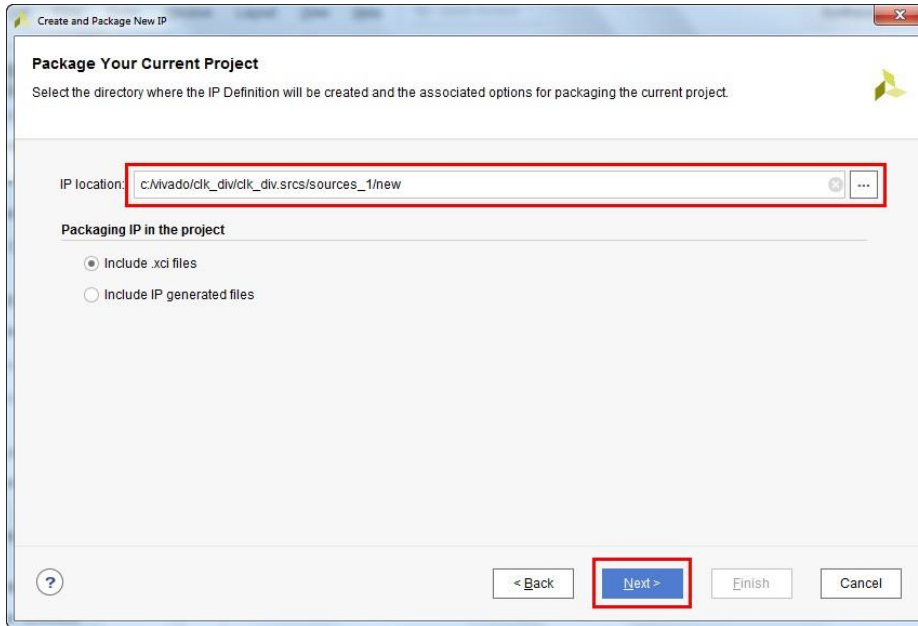
3) 在弹出的对话框中点击‘Next’继续。



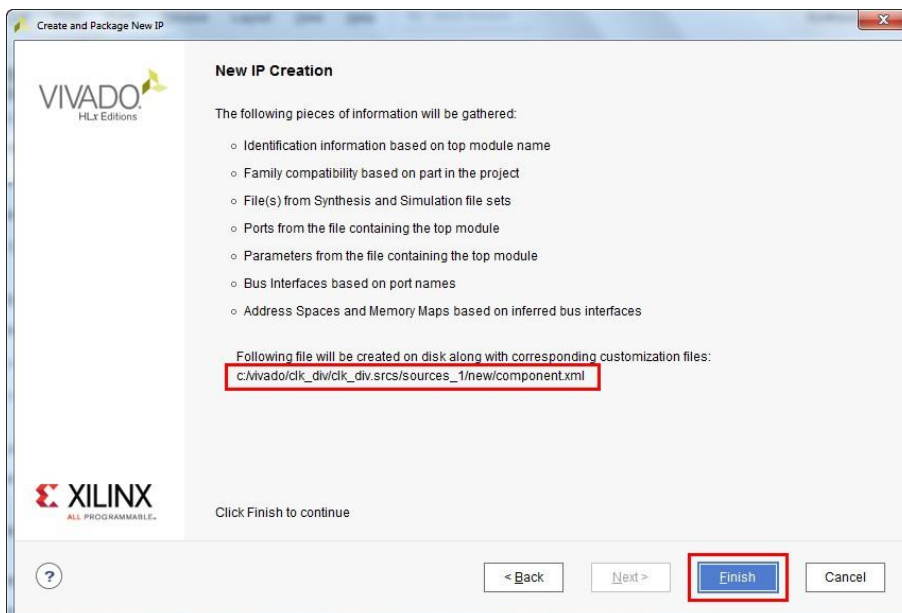
4) 选择‘Package your current project’, 点击‘Next’继续。



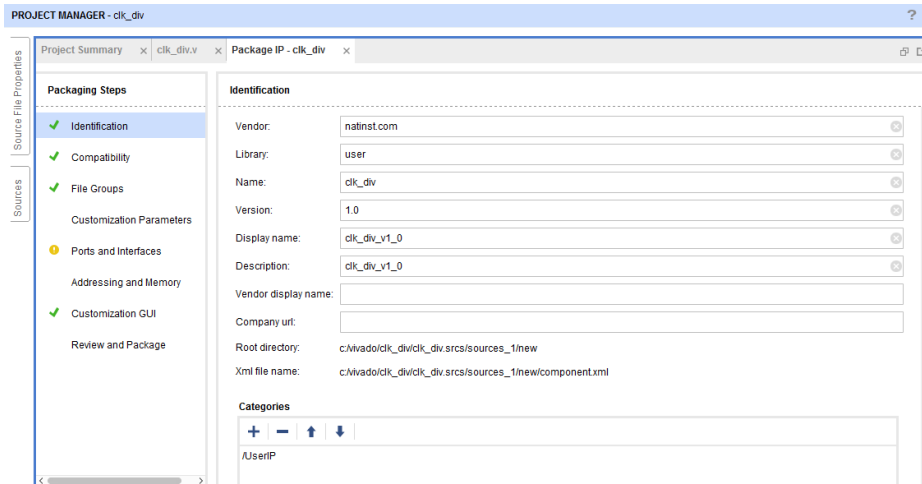
5) 指定创建的新的 IP 核的位置，点击‘Next’继续。



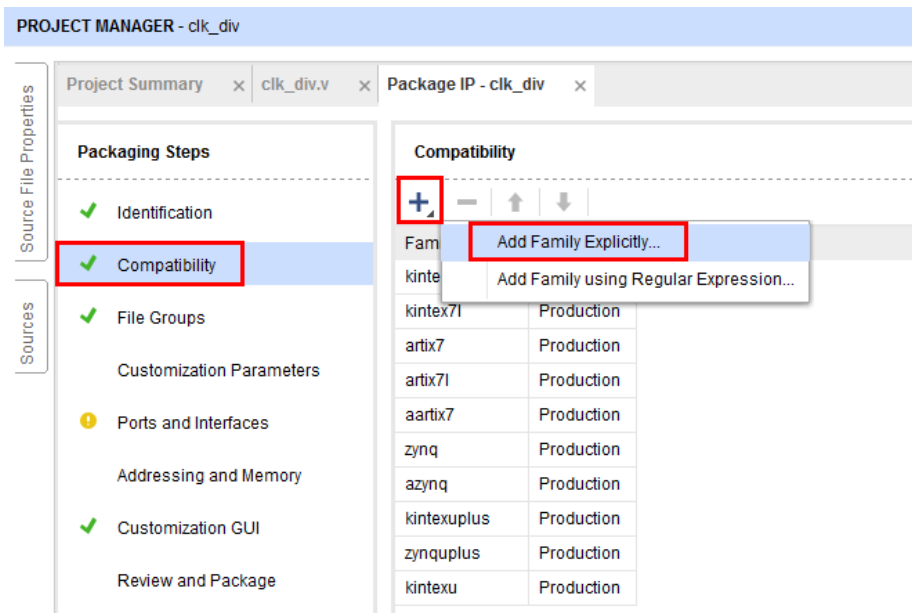
6) 点击 Finish 完成 IP 核的创建。



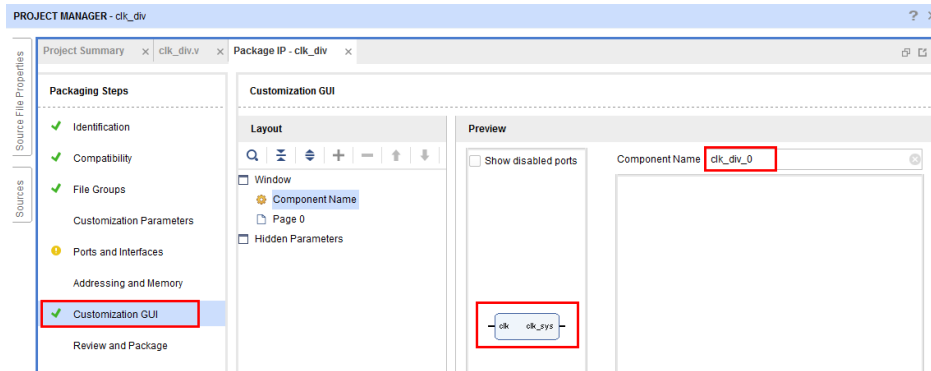
7) 封装完成后，在 Package IP 界面可以查看并修改 IP 信息。



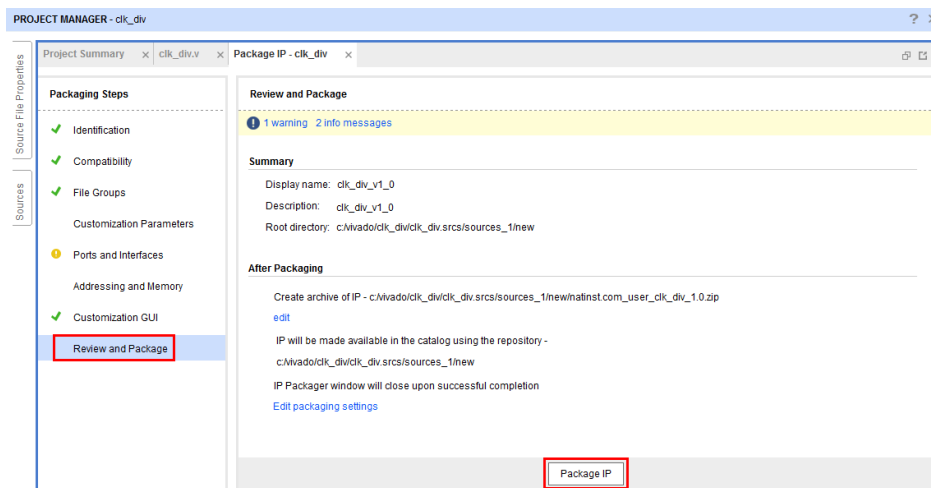
8) 点击‘Compatibility’，添加 IP 适用的产品系列，如果已经添加可以忽略此步。如需添加点击‘+’，选择‘Add Family Explicitly’，勾选相应的产品系列，点击 OK 添加。注：artix7 (Artix-7)必选



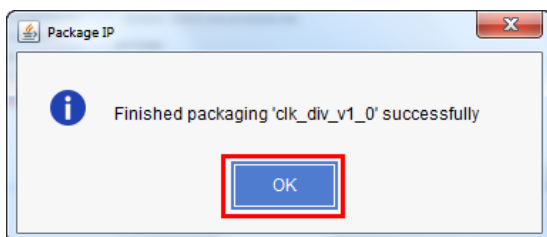
9) 点击‘Customization GUI’，在相应界面可以查看 IP 接口和修改 IP 核名字。



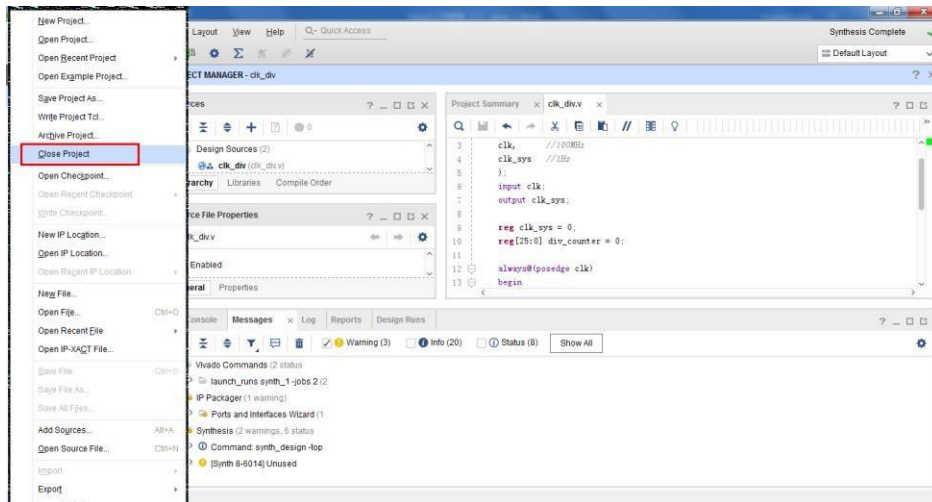
10) 点击‘Review and Package’确认相关信息无误后，点击‘Package IP’开始封装。



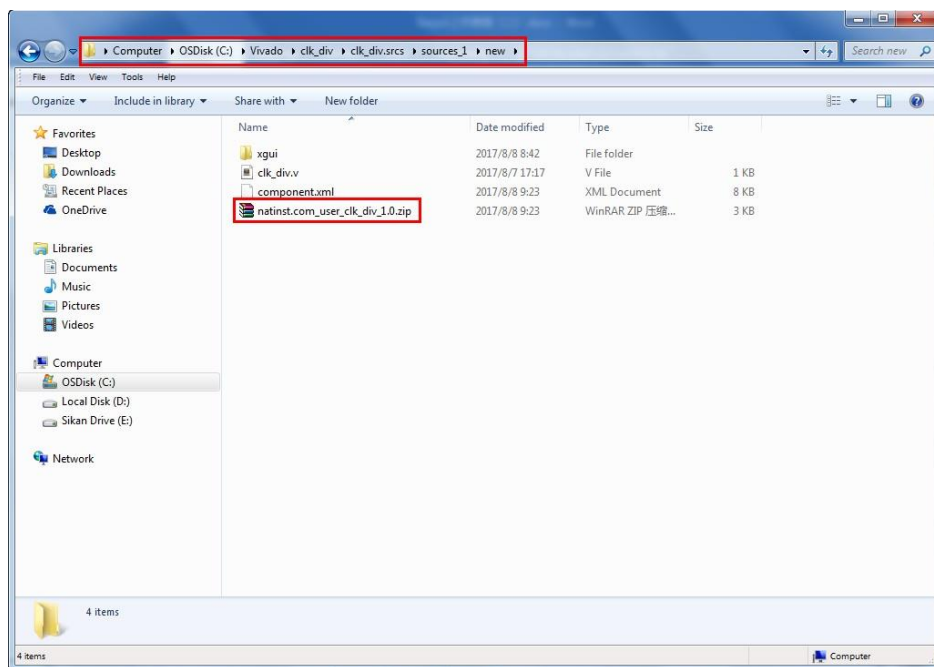
11) 封装完成后弹出提示对话框，点击 OK 继续。



12) 点击工具栏中的‘File’，选择 Close Project 关闭当前工程项目。

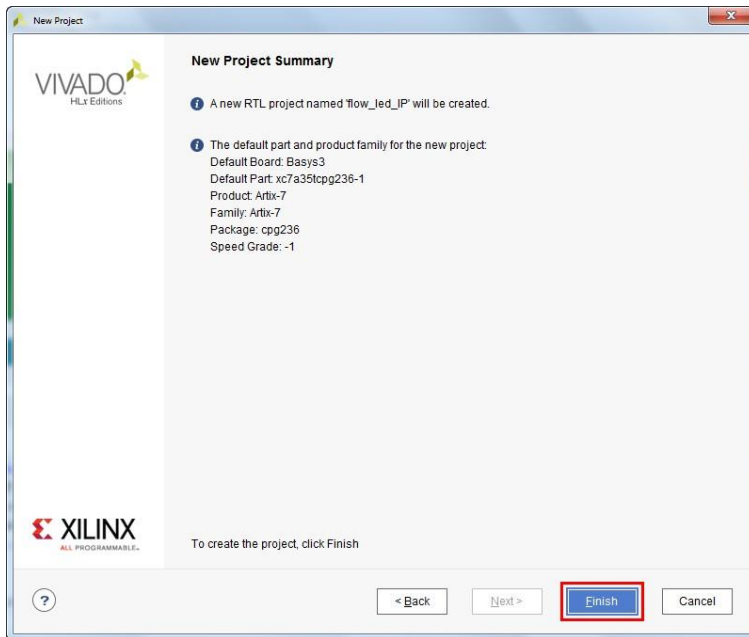


13) 打开资源管理器，进入 IP 封装时指定的根目录可以找到封装好的 IP 核压缩包。

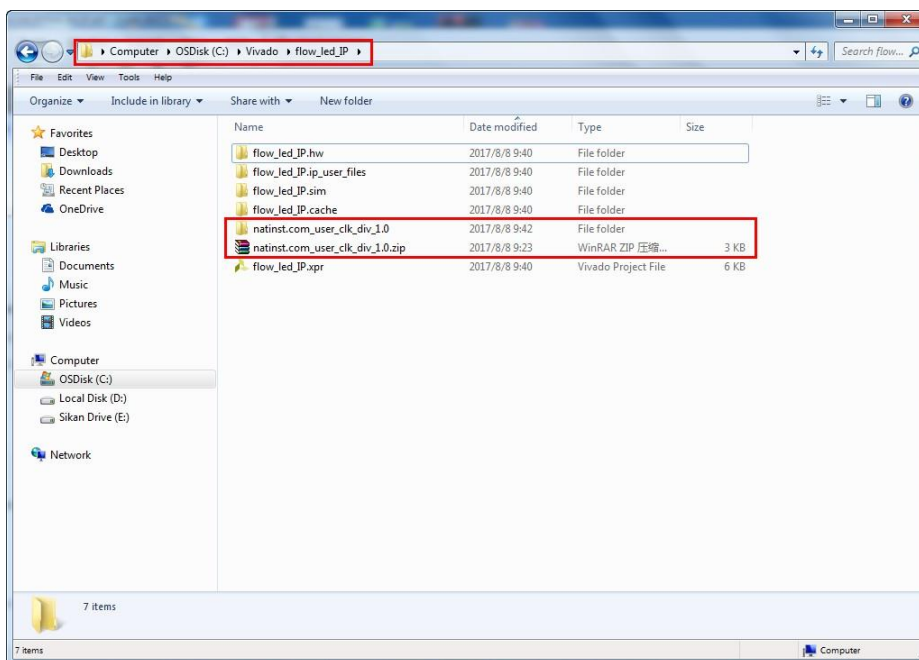


#### 4. 添加用户自定义的 IP 核

1) 新建一个 RTL 工程项目，将新工程命名为‘flow\_led\_IP’。

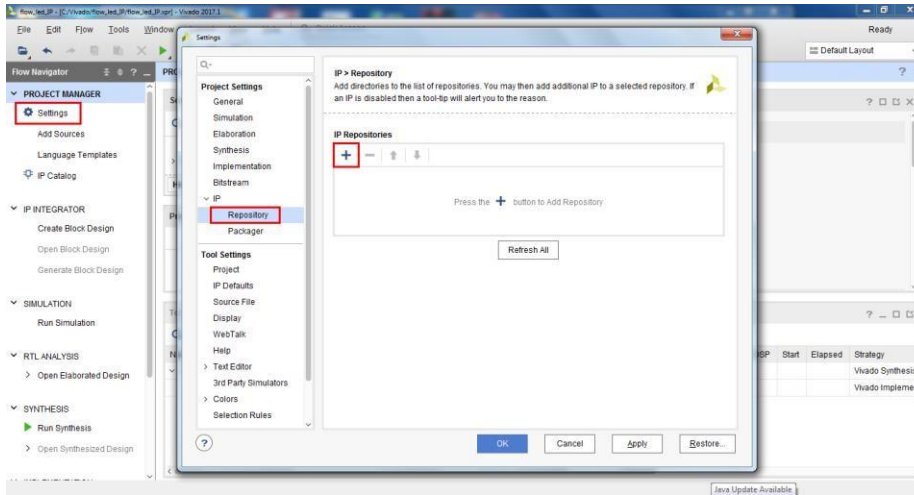


2) 将之前封装的 IP 核压缩包复制到当前创建的工程项目的根目录并解压压缩包。

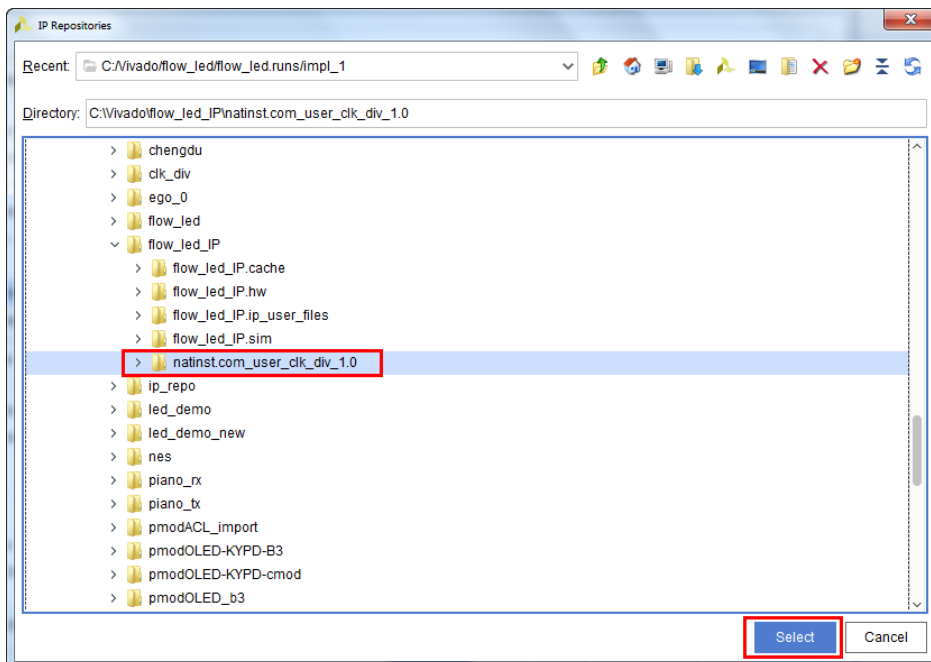




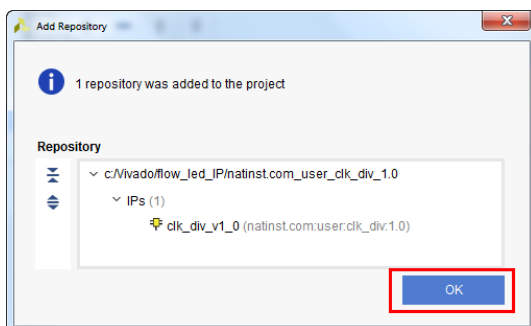
- 3) 在 Flow Navigator 中点击 Settings，展开 IP 一项，选择 Repository， 点击‘+’号添加 IP 库。



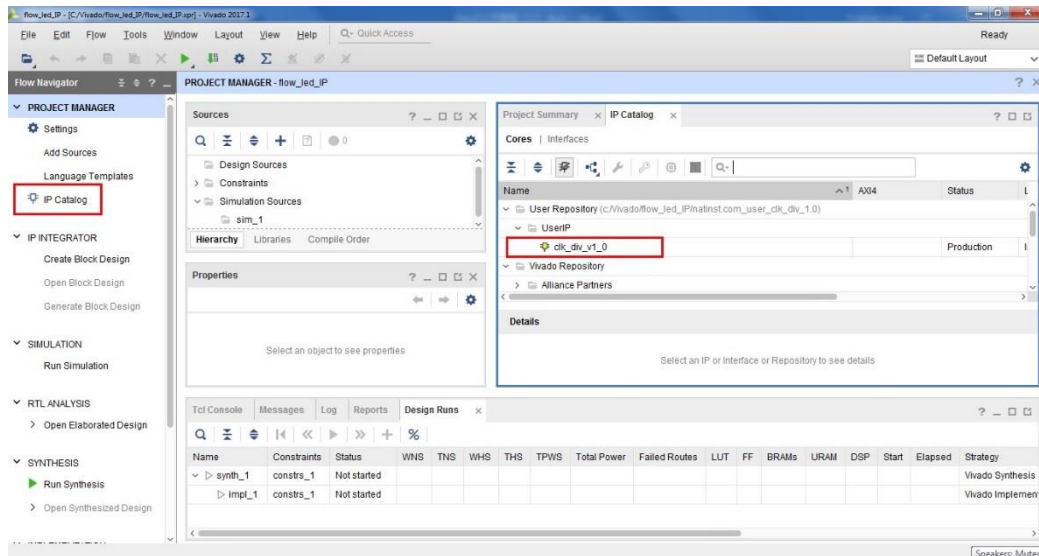
- 4) 选择步骤 2)中解压的文件夹，点击 Select 添加。



- 5) 弹出确认窗口，点击 OK 继续。

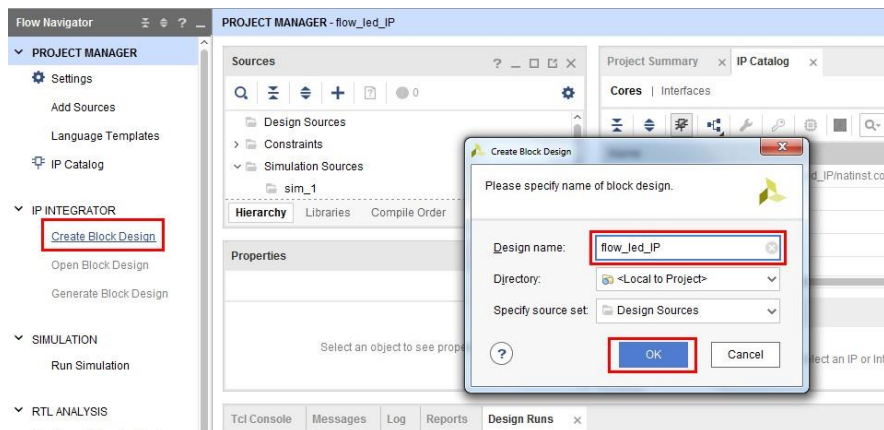


- 6) 在 Settings 界面点击 OK 保存修改并继续。
- 7) 在 Flow Navigator 中点击 IP Catalog, 在 User Repository 展开可以看到添加的用户自定义的 IP 核。

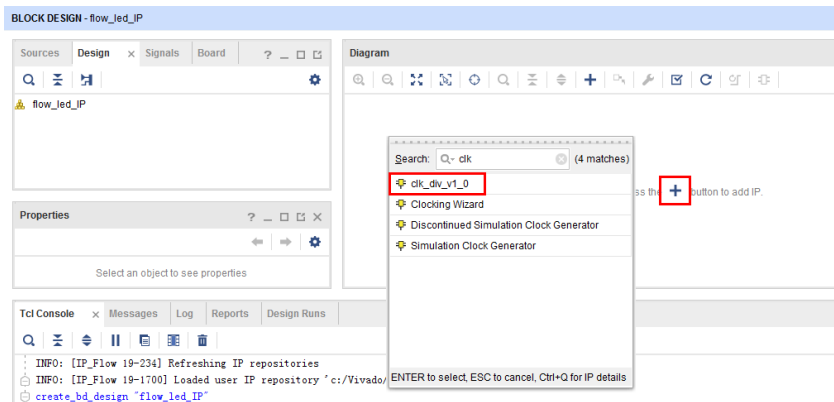


## 5. 模块化设计中调用 IP

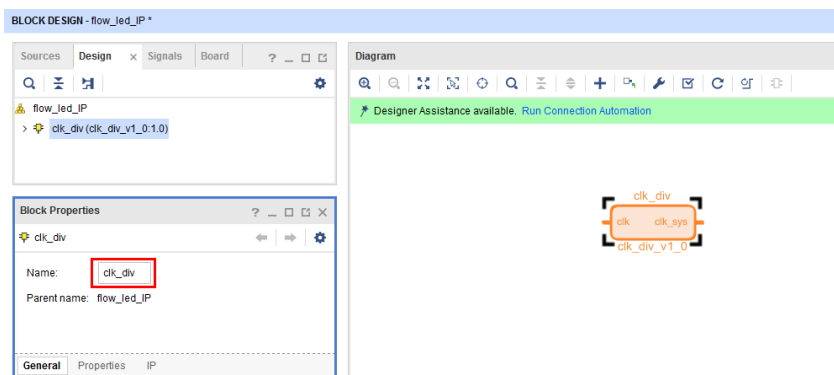
- 1) 在 Flow Navigator 中展开 IP Integrator, 点击‘Create Block Design’。将设计命名为‘flow\_led\_IP’, 点击 OK 完成创建。



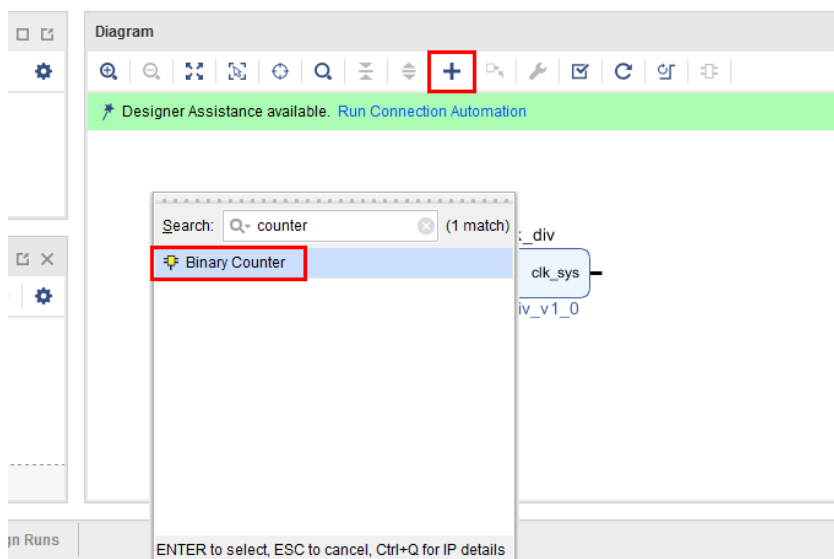
- 2) 在 Diagram 界面，点击中间的‘+’号添加 IP，在搜索栏输入‘clk’， 选择自定义的 IP 核‘clk\_div\_v1\_0’， 按回车添加。



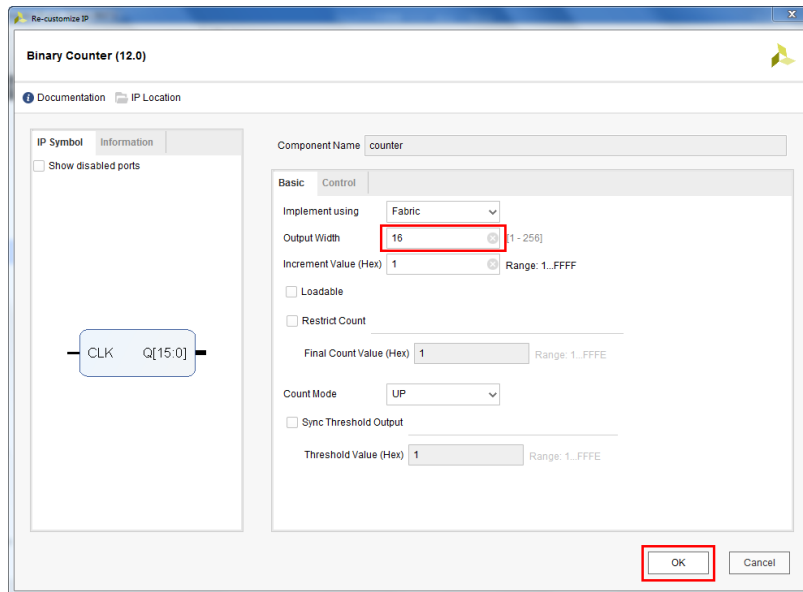
- 3) 选中 Diagram 中添加的 IP 核，在 Block Properties 中将模块名修改为‘clk\_div’。



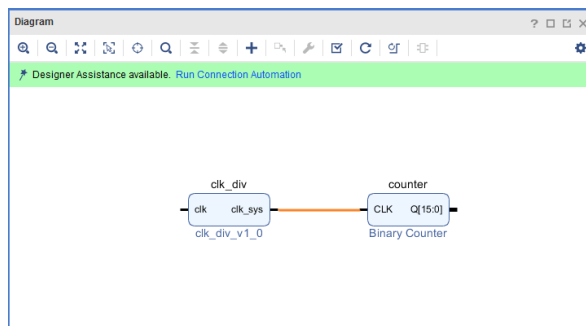
- 4) 在 Diagram 界面中点击中间的‘+’号添加 IP，在搜索框中输入 counter 选择 ‘Binary Counter’， 按回车添加。



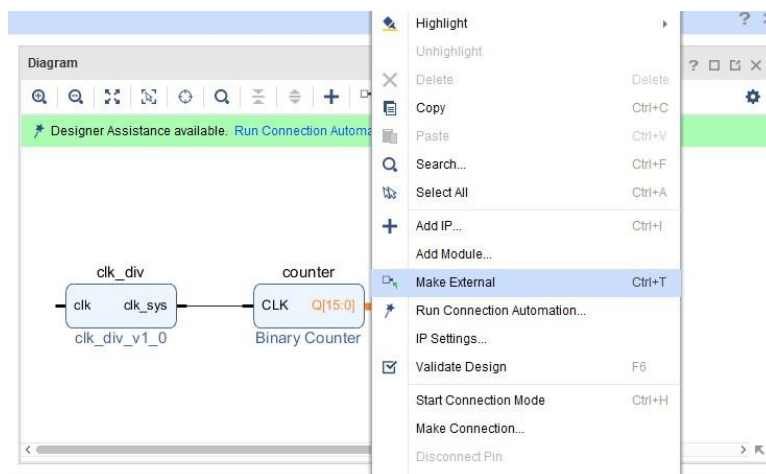
- 5) 将新添加的 Binary Counter 模块重命名为‘counter’。双击模块对模块进行设置，确认 Basic 一项中 Output Width 为 16，点击 OK。



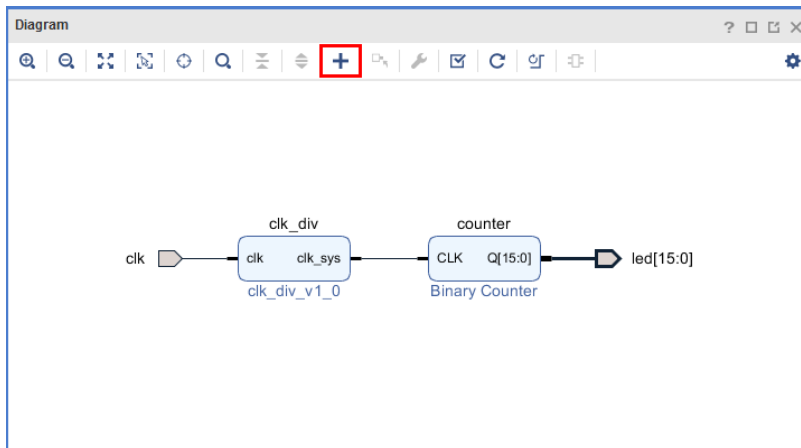
- 6) 将 clk\_div 模块中的 clk\_sys 端口和 counter 模块中 CLK 端口连接起来。



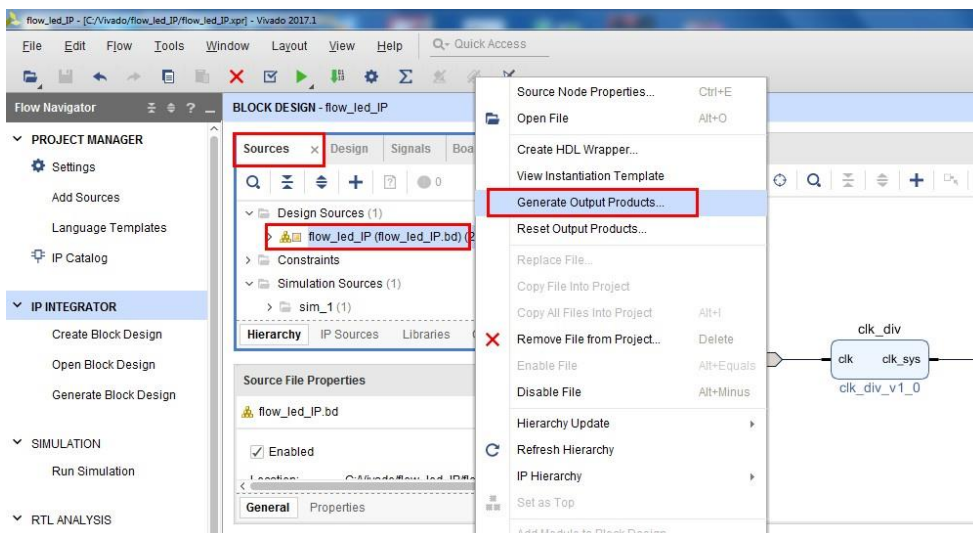
- 7) 点击 counter 模块中 Q[15:0]端口，当变成橙色时右击鼠标，选择‘Make External’。在 External Port Properties 中将端口重命名为 ‘led’。



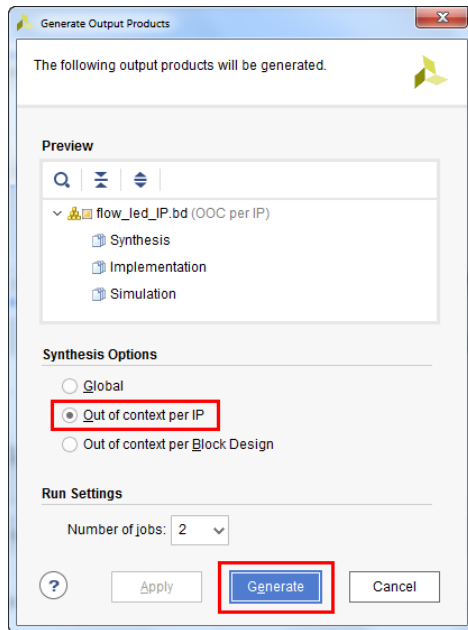
- 8) 同样的，在 `clk_div` 模块中选择 `clk` 端口，右击选择 **Make External**。在 **Diagram** 界面中点击刷新符号重新布局。



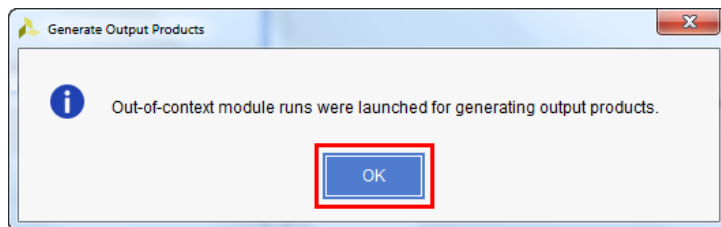
- 9) 在工具栏中点击保存图标保存设计或者使用 **Ctrl+S** 保存。  
10) 在 **Sources** 一项中，展开 **Design Sources**，右击 'flow\_led\_IP'，选择 'Generate Output Products'。



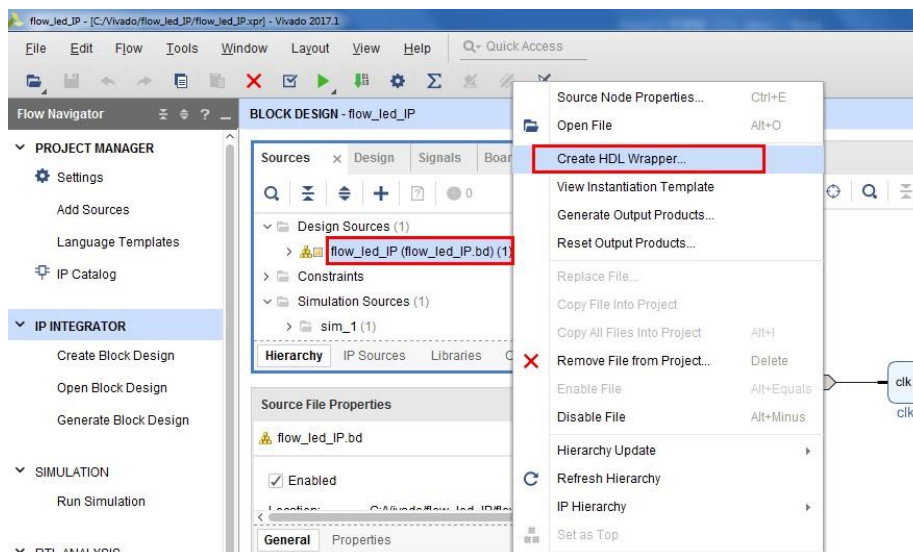
11) 弹出窗口中，在 Synthesis Options 一项中选择‘Out of context per IP’，然后点击 Generate。



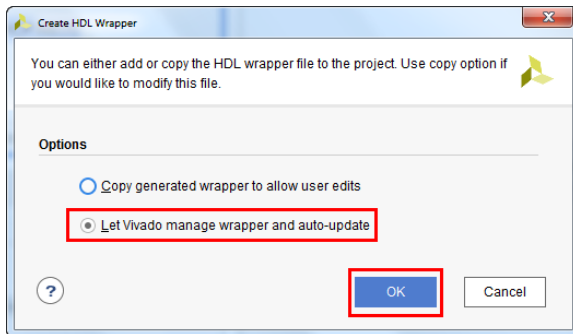
12) 完成后，点击 OK 继续。



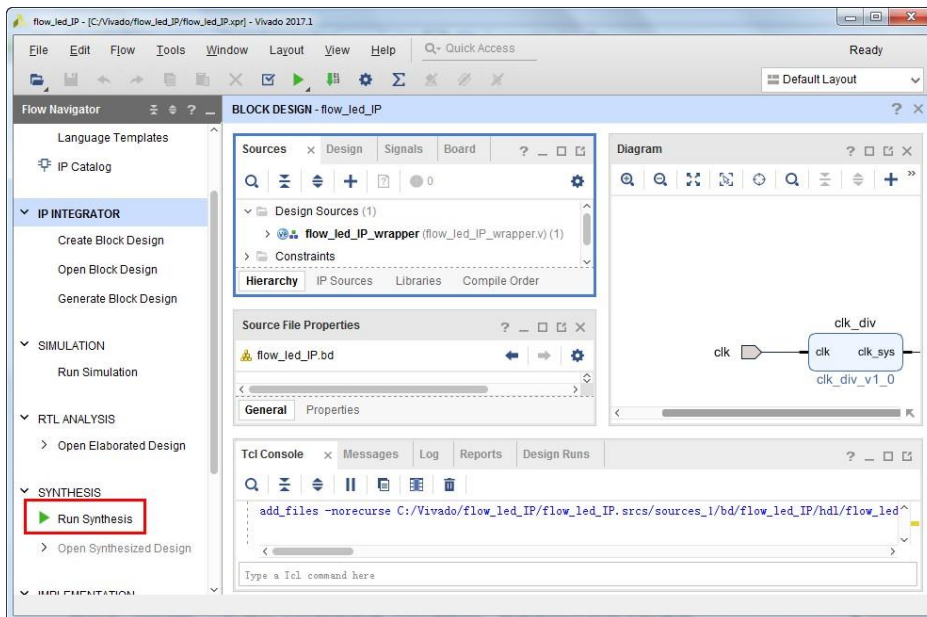
13) 在 Design Sources 中右击 flow\_led\_IP，选择‘Create HDL Wrapper’。



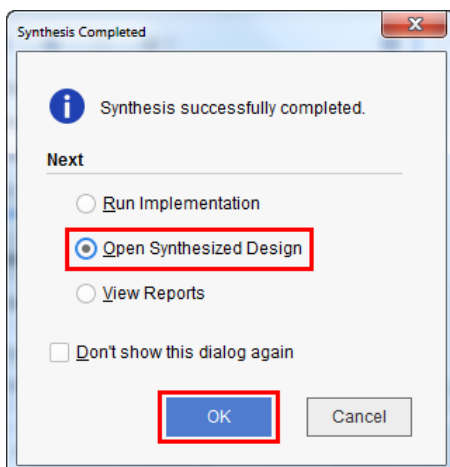
14) 在弹出窗口中选择‘Let Vivado manage wrapper and auto-update’，点击 OK 继续。



15) 在 Flow Navigator 中点击‘Run Synthesis’开始综合。

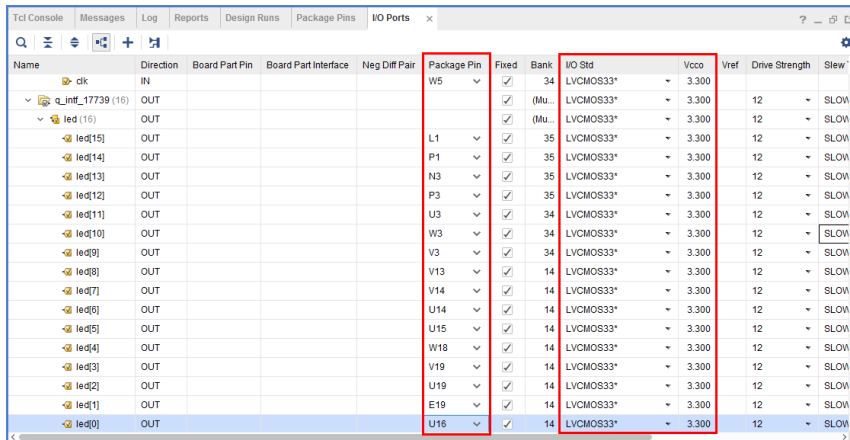


16) 综合完成后，选择‘Open Synthesized Design’查看综合结果。



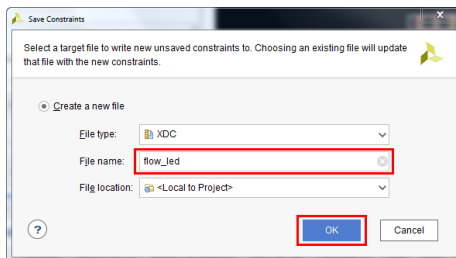


17) 在 I/O Ports 一项中添加引脚约束，将 I/O Std(引脚电平标准)设置为 LVCMOS33(3.3V)。在 Package Pin 一栏中选择相应的引脚位置信息。

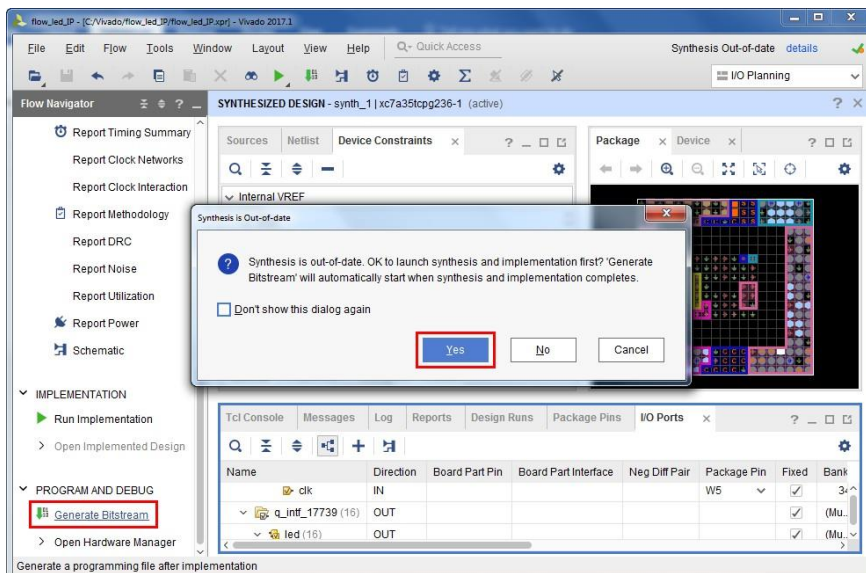


| Name              | Direction | Board Part Pin | Board Part Interface | Neg Diff Pair | Package Pin | Fixed | Bank    | I/O Std   | Vcco  | Vref | Drive Strength | Slew |
|-------------------|-----------|----------------|----------------------|---------------|-------------|-------|---------|-----------|-------|------|----------------|------|
| clk               | IN        |                |                      |               | W5          | ✓     | 34      | LVCMOS33* | 3.300 |      |                |      |
| q_intf_17739 (16) | OUT       |                |                      |               |             | ✓     | (Mu...) | LVCMOS33* | 3.300 | 12   | SLOW           |      |
| led (16)          | OUT       |                |                      |               |             | ✓     | (Mu...) | LVCMOS33* | 3.300 | 12   | SLOW           |      |
| led[15]           | OUT       |                |                      |               | L1          | ✓     | 35      | LVCMOS33* | 3.300 | 12   | SLOW           |      |
| led[14]           | OUT       |                |                      |               | P1          | ✓     | 35      | LVCMOS33* | 3.300 | 12   | SLOW           |      |
| led[13]           | OUT       |                |                      |               | N3          | ✓     | 35      | LVCMOS33* | 3.300 | 12   | SLOW           |      |
| led[12]           | OUT       |                |                      |               | P3          | ✓     | 35      | LVCMOS33* | 3.300 | 12   | SLOW           |      |
| led[11]           | OUT       |                |                      |               | U3          | ✓     | 34      | LVCMOS33* | 3.300 | 12   | SLOW           |      |
| led[10]           | OUT       |                |                      |               | W3          | ✓     | 34      | LVCMOS33* | 3.300 | 12   | SLOW           |      |
| led[9]            | OUT       |                |                      |               | V3          | ✓     | 34      | LVCMOS33* | 3.300 | 12   | SLOW           |      |
| led[8]            | OUT       |                |                      |               | V13         | ✓     | 14      | LVCMOS33* | 3.300 | 12   | SLOW           |      |
| led[7]            | OUT       |                |                      |               | V14         | ✓     | 14      | LVCMOS33* | 3.300 | 12   | SLOW           |      |
| led[6]            | OUT       |                |                      |               | U14         | ✓     | 14      | LVCMOS33* | 3.300 | 12   | SLOW           |      |
| led[5]            | OUT       |                |                      |               | U15         | ✓     | 14      | LVCMOS33* | 3.300 | 12   | SLOW           |      |
| led[4]            | OUT       |                |                      |               | W18         | ✓     | 14      | LVCMOS33* | 3.300 | 12   | SLOW           |      |
| led[3]            | OUT       |                |                      |               | V19         | ✓     | 14      | LVCMOS33* | 3.300 | 12   | SLOW           |      |
| led[2]            | OUT       |                |                      |               | U19         | ✓     | 14      | LVCMOS33* | 3.300 | 12   | SLOW           |      |
| led[1]            | OUT       |                |                      |               | E19         | ✓     | 14      | LVCMOS33* | 3.300 | 12   | SLOW           |      |
| led[0]            | OUT       |                |                      |               | U16         | ✓     | 14      | LVCMOS33* | 3.300 | 12   | SLOW           |      |

18) 点击工具栏中的保存按钮或使用 **Ctrl+S** 保存，在弹出的提示窗中点击 **OK** 继续。在弹出的新建约束文件对话框中输入‘flow\_led’，点击 **OK** 创建并保存约束文件。

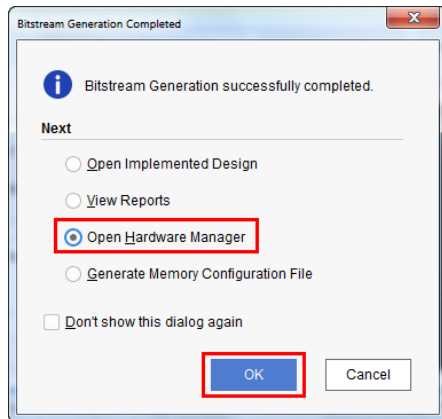


19) 在 Flow Navigator 中，展开 Program and Debug，点击 Generate Bitstream。由于添加了约束文件，弹出的提示窗提示需要重新综合，点击‘Yes’继续。Vivado 会依次执行综合(Synthesis)，实现(Implementation)和生成比特流文件。

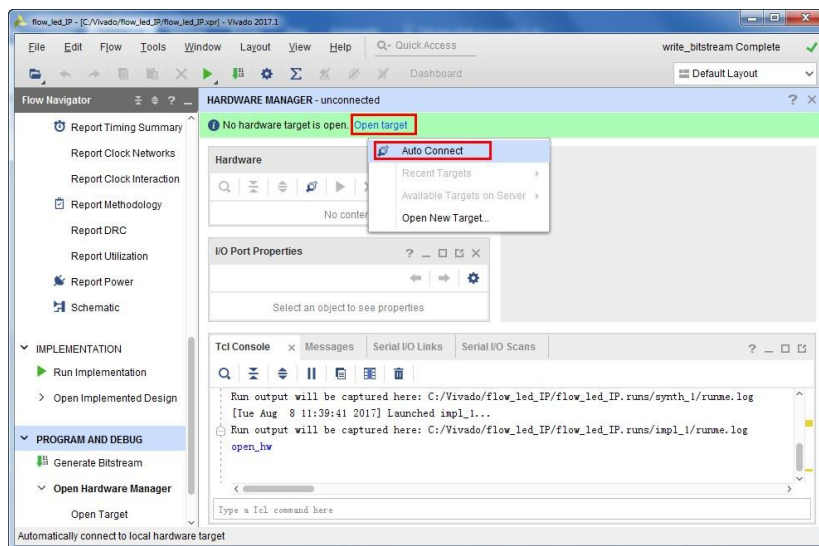




20) 完成之后选择‘Open Hardware Manager’，连接 Basys 3 FPGA 开发板和电脑。



21) 打开 Basys 3 FPGA 电源，点击‘Open target’，选择‘Auto Connect’。



22) 完成连接后，点击‘Program device’将比特文件下载到 Basys3 开发板上运行。Basys3 板上运行演示如下所示。

